

TP2 : ENCAPSULATION

Objectif

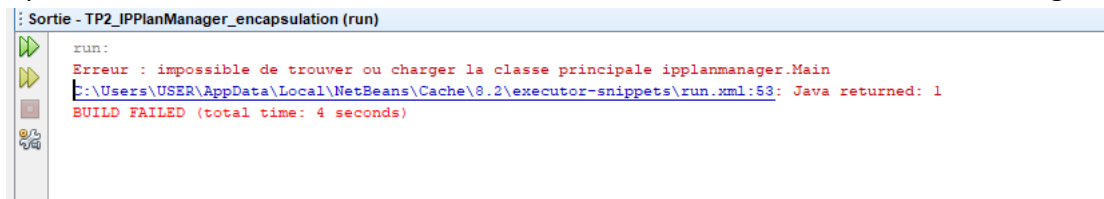
Introduction de l'encapsulation et des validations.

Notions étudiées

- private
- getters
- setters
- validation
- this

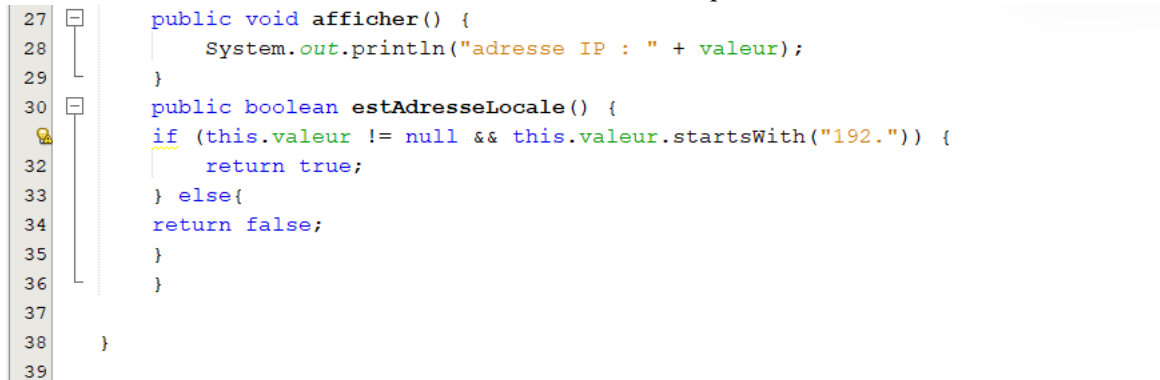
Tests réalisés

Après avoir modifié la classe main, nous l'avons exécuté et avons obtenu message d'erreur



```
Sortie - TP2_IPPlanManager_encapsulation (run)
run:
Erreur : impossible de trouver ou charger la classe principale ipplanmanager.Main
C:\Users\USER\AppData\Local\NetBeans\Cache\8.2\executor-snippets\run.xml:53: Java returned: 1
BUILD FAILED (total time: 4 seconds)
```

1. Ajouter dans la classe AdresseIP une méthode : public boolean estAdresseLocale() Cette méthode devra retourner : true si l'adresse commence par : 192.



```
27 public void afficher() {
28     System.out.println("adresse IP : " + valeur);
29 }
30 public boolean estAdresseLocale() {
31     if (this.valeur != null && this.valeur.startsWith("192.")) {
32         return true;
33     } else {
34         return false;
35     }
36 }
37
38 }
39
```

2. Ajout d'équipements
 - un switch
 - un pc client

```

public static void main(String[] args) {
    System.out.println("==== TP2 : Encapsulation =====");
    addresseesIP ip1 = new addresseesIP("192.168.1.1");
    addresseesIP ip2 = new addresseesIP("");
    addresseesIP ip3 = new addresseesIP(null);
    addresseesIP ip4 = new addresseesIP("192.168.1.4");
    addresseesIP ip5 = new addresseesIP("192.168.1.5");
    addresseesIP ip6 = new addresseesIP("");
    InterfaceReseau interface1 = new InterfaceReseau("eth0", ip1);
    InterfaceReseau interface2 = new InterfaceReseau("", ip2);
    InterfaceReseau interface4 = new InterfaceReseau("eth0", ip4);
    InterfaceReseau interface5 = new InterfaceReseau("eth1", ip5);

    interface1.activer();
    interface4.activer();
    interface5.activer();
    Equipement routeur = new Equipement(
        "R1_EDGE",
        "Routeur",
        interface1
    );
    Equipement Switch = new Equipement(
        "sw_1",
        "Switch",
        interface4
    );
    Equipement pc_client1 = new Equipement(
        "pc_1",
        "client",
        interface5
    );
    Equipement serveur =
        new Equipement(
            "",
            "",
            interface2
        );
}

```

Difficultés rencontrées

probleme de compilation

```

: Sortie - TP2_IPPlanManager_encapsulation (run)
==== TP2 : Encapsulation =====
Erreur : adresse IP invalide.
Exception in thread "main" java.lang.NoClassDefFoundError: ipplanmanager/InterfaceReseau
Erreur : adresse IP invalide.
    at ipplanmanager.Main.main(Main.java:19)
Caused by: java.lang.ClassNotFoundException: ipplanmanager.InterfaceReseau
    at java.net.URLClassLoader.findClass(URLClassLoader.java:391)
    at java.lang.ClassLoader.loadClass(ClassLoader.java:424)
    at sun.misc.Launcher$AppClassLoader.loadClass(Launcher.java:338)
    at java.lang.ClassLoader.loadClass(ClassLoader.java:357)
    ... 1 more
C:\Users\USER\AppData\Local\NetBeans\Cache\8.2\executor-snippets\run.xml:53: Java returned: 1
BUILD FAILED (total time: 0 seconds)

```

Réponses aux questions

1. Pourquoi utilise-t-on `private` dans les classes ?

On utilise le modificateur `private` pour respecter le principe d'**encapsulation**. Cela permet de cacher les détails internes d'un objet et d'empêcher que ses données ne soient modifiées de manière anarchique ou accidentelle depuis l'extérieur de la classe.

2. Quelle différence existe entre un attribut public et un attribut privé ?

- Un attribut **public** est accessible et modifiable par n'importe quelle autre classe du programme.
- Un attribut **privé** n'est accessible que par les méthodes de la classe où il est déclaré. Pour le manipuler de l'extérieur, il faut obligatoirement passer par des interfaces contrôlées (méthodes).

3. Pourquoi utilise-t-on des getters et setters ?

Ils servent d'intermédiaires sécurisés. Le **getter** permet de lire une valeur privée, tandis que le **setter** permet de la modifier tout en y ajoutant des **règles de contrôle** (par exemple, vérifier qu'une note est bien comprise entre 0 et 20).

4. Pourquoi les validations sont-elles importantes dans un logiciel réseau ?

En réseau, les données reçues peuvent être corrompues, mal formatées ou provenir d'attaques. Les validations garantissent que le logiciel ne traite que des données cohérentes (adresses IP valides, numéros de ports corrects), évitant ainsi des erreurs de segmentation ou des failles de sécurité.

5. Quel est le rôle du mot-clé `this` ?

Le mot-clé `this` désigne l'**instance courante** de l'objet. Il est principalement utilisé pour lever l'ambiguïté lorsque le nom d'un paramètre (dans un constructeur ou une méthode) est identique au nom d'un attribut de la classe.

6. Pourquoi le constructeur appelle-t-il les setters ?

Appeler les setters dans le constructeur permet de réutiliser la **logique de validation** déjà écrite. Cela garantit que les données fournies lors de la création de l'objet sont vérifiées immédiatement, empêchant la création d'un objet avec des données invalides.

7. Pourquoi la validation du masque CIDR est-elle importante ?

Le masque CIDR détermine la structure du réseau. Une valeur erronée (en dehors de 0-32 pour IPv4) rendrait les calculs d'adresses réseau impossibles ou faux, ce qui pourrait bloquer toute communication ou créer des vulnérabilités dans la segmentation.

8. Pourquoi l'encapsulation améliore-t-elle la sécurité logicielle ?

L'encapsulation réduit la "surface d'attaque" d'un objet. En ne laissant l'accès aux données que via des méthodes spécifiques et vérifiées, on empêche les comportements imprévus et les manipulations malveillantes de l'état interne du système.